

PALMPAY CLONE - COMPLETE CODE GUIDE

=====

This file explains every part of the app, how the logic flows, and exactly where to go to change anything. The app is Java + XML (no Kotlin, no Jetpack Compose), organised with OOP principles: each screen has an Activity (shell) plus a Controller (behaviour), data lives in repositories/stores, and screens are described by immutable model objects built in catalogues.

0. BIG PICTURE

Packages (app/src/main/java/damjay/palmpay/clone/):

.	MainActivity	Home screen shell.
.ui	HomeScreenController	All home behaviour.
.data	WalletStore	Persisted user data (balance, display name, Paystack key).
	HomeCatalog	Static home content (models).
.model	QuickAction, ServiceAction, PromotionCard	Immutable value objects.
.profile	ProfileActivity / ProfileScreenController	Customisation form.
.transfer.ui	TransferActivity, TransferScreenController, AmountActivity, AmountScreenController, BankPickerActivity, BankPickerScreenController, BankPickerDialog, BankListAdapter	
.transfer.data	BankDirectoryRepository (online bank list), PaystackClient (name resolution), NubanBankResolver (check-digit math), BankNameNormalizer (dedupe/aliases), BankLogoLoader (rounded logo cache), BankLogoResolver (local logo mapping), TransferRepository (interface) + LocalTransferRepository + TransferCatalog	(demo transfer history).
.transfer.model	BankInstitution, TransferRecipient, TransferShortcut	

Layouts (app/src/main/res/layout/): one XML per screen + one per list row.

Values (res/values + res/values-night): colors/strings per theme.

Drawables: res/drawable (XML shapes/vectors), res/drawable-nodpi (PNG art).

Design pattern per screen (Model-View-Controller style):

- * The Activity only inflates its ViewBinding, wires system bars, and owns the Controller. No business decisions live in Activities.
- * The Controller receives (Context, Binding) in its constructor, and its bind() method attaches listeners and renders models into views.
- * Catalogues/Repositories are static or injected sources of model objects, so swapping demo data for a real API later only touches those classes.

1. HOME SCREEN

Files: MainActivity.java, ui/HomeScreenController.java, layout/activity_main.xml, data/HomeCatalog.java, data/WalletStore.java

Flow:

MainActivity.onCreate ->	HomeScreenController.bind() which calls, in order:
renderQuickActions()	4 cards (To Bank / To PalmPay / Savings / Cards). To Bank click -> TransferActivity.start(context).
renderServices()	8-tile grid (Airtime..More). The Data tile shows an orange "Promo" badge (service_badge view).
renderPromotions()	"Team Up & Save" + "CashBox" cards (promo pages).
bindBalanceCard()	Shows WalletStore.getBalanceDisplay(); eye button toggles hide/show; Add Money / History = toasts.
bindHeader()	Greeting "Hi, <name>" from WalletStore display name; profile button -> ProfileActivity.
bindClaimCard()	Claim button toast.
bindPromoCarousel()	ViewFlipper (id promo_flipper) auto-flips every 4000 ms with fade animations; dots (claim_dots) are re-coloured by a Handler loop in sync.
bindBannerCarousel()	ViewFlipper (id banner_flipper) rotating the three bottom banners (borrow/verify/business), 4000 ms.
bindNavigation()	Builds the custom bottom bar (bottom_bar) from bottom_nav_item.xml rows: icon mask tinted purple when selected else ink, label, red dot on Loan, NEW pill on Wealth.

Where to edit:

- * Text of any tile/card res/values/strings.xml
- * Which tiles exist / their icons .. data/HomeCatalog.java
- * Behaviour on tap ui/HomeScreenController.java
- * Carousel speed the two setFlipInterval(4000) calls
- * Colours (light/dark) res/values/colors.xml and res/values-night/colors.xml

2. PROFILE (customisation form)

Files: profile/ProfileActivity.java, profile/ProfileScreenController.java, layout/activity_profile.xml, data/WalletStore.java

Three editable fields, one Save button:

- * Balance (numberDecimal) -> WalletStore.saveBalance()
- * Display name -> WalletStore.saveDisplayName() (drives the home greeting)
- * Paystack API key -> WalletStore.savePaystackApiKey()

Everything is stored in SharedPreferences "palmpay_clone_wallet".

Home re-reads the balance in MainActivity.onResume() when you return.

Where to edit: add a field = activity_profile.xml + controller saveAll().

3. TRANSFER TO BANK (the big one)

Files: transfer/ui/TransferActivity.java (shell + screen transitions),
transfer/ui/TransferScreenController.java (ALL logic, ~800 lines),
layout/activity_transfer.xml.

3.1 Account input

- * A TextWatcher formats digits as "911 241 3798" (formatAccountNumber) and after every change calls updateAccountMatch(digits).
- * < 10 digits: renders history suggestions (renderAccountSuggestions) with progressive purple prefix colouring (applyProgressiveAccountColor using SpannableString + ForegroundColorSpan). Any edit also CLEARS all verification state (resolvedNames, failedBankKeys, listedBankNames) so nothing is ever persisted between requests.
- * == 10 digits: hides suggestions, shows the matching-banks flow below.

3.2 Matching banks request cycle (showMatchingBanks)

1. Clears the list container, shows the "Matching banks" spinner row (bank_status_row).
2. Adds transfer-history banks for this account instantly (they provably work) with their saved holder names.
3. With a Paystack key (WalletStore.getPaystackApiKey()):
PaystackClient.listBanks() -> the GENERAL bank directory, served from memory -> disk cache -> network (this is the only persisted thing besides logos).
Builds probe targets = wallet providers (WALLET_PROVIDERS: opay, palmpay, moniepoint, smartcash, kuda, momo) + up to MAX_VERIFIED_PROBES (10) NUBAN-valid banks (NubanBankResolver.candidateBanks).
Fires ONE resolveAccount() per target, ALL CONCURRENTLY (OkHttp dispatcher 64 requests / 40 per host).
Each success -> addMatchingBankRow() (row appears immediately and its name is cached in the in-memory resolvedNames map for THIS request cycle only). Each failure -> remembered in failedBankKeys so picking that bank later never spins uselessly.
When the last probe settles -> finishMatchingList() hides spinner.
4. Without a key: offline fallback lists NUBAN candidates from the GitHub-pages directory (BankDirectoryRepository) with no names.
Important policy (user requirement): verification results are NEVER persisted. Deleting and retyping a digit always re-runs the whole request.

3.3 Selecting a bank

- * Row click -> selectMatchedBank(): fills Bank field (logo via applyBankLogo), then:
 - name already in resolvedNames -> shown instantly, Next enables;
 - bank known-failed this cycle -> red "Invalid account" banner;
 - otherwise -> "Verifying account name" spinner + resolveNameViaPaystack() (history checked first).
- * Picker selection -> selectBank(): same cache/failed/verify ladder.
- * resolveNameViaPaystack(): PaystackClient.listBanks(), match the bank by canonical name, resolveAccount(digits, bank) with snake_case params (account_number, bank_code). Success -> confirmation row with holder name + Next enabled. Failure -> invalid-account banner.

3.4 Next gating

isFormReady() = 10 digits AND a bank selected AND a resolved name (trustedRecipient or resolvedRecipient). The Next button is silent when not ready (no toast).

3.5 Support classes you will touch

- * NubanBankResolver.java - pure static CBN check-digit algorithm (isValidNuban, candidateBanks). Unit-tested with CBN circular vectors.
- * BankNameNormalizer.java - normalize()/canonical()/dedupe(): strips LIMITED/MFB/PSB/etc., aliases variants (GTBank<->Guaranty Trust...), dedupes directory lists by code then canonical name.
- * BankLogoResolver.java - localLogoFor()/fallbackForProvider(): canonical name -> bundled rounded logo resource.
- * BankLogoLoader.java - downloads logos, centre-crops + circle-clips them ONCE (roundLogo), memory LruCache + disk cache (cache/bank_logos/).
- * BankDirectoryRepository.java - fetches Banks.json (GitHub pages), fallbackBanks() offline; results deduped via BankNameNormalizer.
- * PaystackClient.java - OkHttp client; listBanks() (cached general list) and resolveAccount(); NO persistence of verification results.

Where to edit:

- * Probe count / wallet set TransferScreenController constants
- * Strings (Matching banks, Verifying..., Invalid account...) = strings.xml
- * Banner/row layouts matching_bank_item.xml, activity_transfer.xml

4. BANK PICKER (Select bank)

Files: BankPickerActivity.java, BankPickerScreenController.java, BankListAdapter.java, layout/activity_bank_picker.xml + rows.

- * Shows "Frequently Used Bank" grid; when an account number is passed in (TransferActivity.openBankPicker(digits)) and it has NUBAN-verified matches, those verified banks top the grid; otherwise a fixed set.
- * Alphabetical full list from BankDirectoryRepository (online, deduped), section headers + A-Z index rail; search filters by name/code.
- * Logos via BankLogoLoader (rounded + cached).
- * Result returns to TransferScreenController.selectBank().

5. AMOUNT SCREEN ("Insert Value")

Files: AmountActivity.java, AmountScreenController.java, layout/activity_amount.xml.

- * Recipient header (logo/name/account/provider); logo URL travels via TransferRecipient.logoUrl when known, else BankLogoResolver fallback.
- * Amount entry uses the CUSTOM KEYPAD (GridLayout id amount_keypad). System keyboard is suppressed for the amount field (setShowSoftInputOnFocus(false) + hideSystemKeyboard() on focus).
- * Focus choreography: focusing the NOTE field hides the custom keypad and allows Gboard; leaving Note restores the keypad and hides Gboard. Manifest uses adjustResize so content scrolls above the keyboard.
- * Input is comma-grouped live (formatAmount); parseAmount strips commas.
- * Validation: 10.00 - 200,000.00; out-of-range shows the red amount_error text and disables Next (bg_keypad_next vs bg_keypad_next_enabled); >= 10,000 shows the FIRS stamp-duty notice (stamp_notice_card, 13sp).
- * Balance line: "Balance: (naira) 0.00 CashBox: <wallet balance>" (balance_cashbox string; Balance is always 0 by product rule).
- * Quick chips 500..10,000 (explicit GridLayout rows/columns).

Where to edit: keypad keys = activity_amount.xml; rules = AmountScreenController constants (MIN_AMOUNT, MAX_AMOUNT, STAMP_DUTY_THRESHOLD).

6. PERSISTENCE POLICY (what is stored on device)

SharedPreferences "palmpay_clone_wallet" (WalletStore):
available_balance, display_name, paystack_api_key [user data]
SharedPreferences "palmpay_clone_paystack" (PaystackClient):
bank_directory_json [GENERAL bank list]
Cache dir bank_logos/ (BankLogoLoader): processed rounded logos.
NOT stored: resolved account names, failed-bank memory, matching lists.
Those live only in TransferScreenController fields for the current request cycle and are wiped on any account edit and on process death.

7. THEMES

res/values (light) and res/values-night (dark) hold the same color names; Android picks per system theme. Drawables in drawable-night/ override for dark. Screens read colors only via @color/... so a theme tweak is a single value edit. The FREE banner, surfaces, inks etc. all follow this.

8. TESTS & CI

app/src/test/java/... :

- HomeCatalogTest / UiModelTest / ServiceActionTest / TransferCatalogTest guard catalogue/model invariants (distinct icons, formatting...).
- NubanBankResolverTest CBN circular examples (011/0000014579 etc.).
- BottomNavigationBarTest (Robolectric) inflates MainActivity and asserts the five bottom-bar labels render.
- PaystackLiveProbeTest @Ignore-d live diagnostic that hit the test key.

CI (.github/workflows/android.yml, user-supplied): runs unit tests then assembleDebug and uploads the APK artifact on every push.

9. CHEAT SHEET - "WHERE DO I CHANGE ...?"

Greeting/name/balance/key storage	... data/WalletStore.java
Home tiles & icons	... data/HomeCatalog.java
Home behaviour/carousels	... ui/HomeScreenController.java
Bottom bar look	... layout/bottom_nav_item.xml + bindNavigation()
Transfer flow (all of it)	... transfer/ui/TransferScreenController.java
Matching-bank request policy	... same file: showMatchingBanks(), WALLET_PROVIDERS, MAX_VERIFIED_PROBES
Name resolution HTTP	... transfer/data/PaystackClient.java
Bank math	... transfer/data/NubanBankResolver.java
Name variants/duplicates	... transfer/data/BankNameNormalizer.java
Logos	... transfer/data/BankLogoLoader.java + BankLogoResolver.java + drawable-nodpi/
Amount rules	... transfer/ui/AmountScreenController.java

Picker transfer/ui/BankPickerScreenController.java
Any user-visible text res/values/strings.xml
Any colour res/values/colors.xml (+ values-night)
Any shape/rounded bg res/drawable/*.xml